

数据驱动对象刷新系统

这是一套可直接接入 Unity 的通用刷新框架。它按毫秒时间轴创建 Prefab，通过反射自动发现/添加任意 Component，再根据目标字段的真实类型转换 JSON 数据并赋值。核心流程中没有任何具体业务组件分支，也没有使用 Resources.Load。

接入

1. 将 Scripts/Spawn 整个目录复制到 Unity 工程的 Assets/Scripts/Spawn。
2. 在场景中新建对象并挂载 DataDrivenSpawn.SpawnController
3. 在 Spawn Text 中放入刷新文本 TextAsset。
4. 将可创建或可被字段引用的全部 Prefab 拖入 Prefab Table。注册键严格使用 prefab.name，名称区分大小写。
5. 业务组件的可配置成员声明为 public field；JSON 的 Name 必须与字段名称完全一致。

刷新文本每行格式如下，前五个分隔符必须是真实 Tab：

```
createTime<Tab>createName<Tab>posX<Tab>posY<Tab>posZ<Tab>json
```

解析器只消费前五个 Tab，剩余内容全部作为 JSON，因此 JSON 内部可以包含 Tab。空行及以 # 开头的行会被忽略。记录会按 createTime 排序，相同时间保持原文件顺序。

JSON 格式

```
{
  "ComponentVar": [
    {
      "Component": "Game.Enemy01Ctrl",
      "Variables": [
        { "Name": "targetPosition", "Data": { "x": 0, "y": 0, "z": 0 } },
        { "Name": "moveSpeed", "Data": 5 },
        { "Name": "bulletPrefab", "Data": "Bullet-001" }
      ]
    }
  ]
}
```

Component 推荐填写包含命名空间的完整类型名。简单类型名也可以使用，但当两个命名空间存在同名组件时，框架会拒绝歧义并输出警告。

已支持 bool、int、long、float、double、string、Vector2、Vector3、Quaternion、Color、GameObject 和所有 Enum。Quaternion 的 x/y/z 按欧拉角解释；Color.a 可省略，默认值为

1; `GameObject` 的字符串值从同一 `Prefab` 注册表查找。

扩展新类型

新增一个实现 `ITypeConverter` 的公开、非抽象、无参构造类即可被框架自动发现。转换器建议添加 `[UnityEngine.Scripting.Preserve]` 避免 IL2CPP 代码裁剪：

```
[UnityEngine.Scripting.Preserve]
public sealed class Vector4TypeConverter : ITypeConverter
{
    public bool CanConvert(Type type)
    {
        return type == typeof(Vector4);
    }

    public object Convert(
        object jsonData,
        Type targetType,
        Dictionary<string, GameObject> prefabDictionary)
    {
        // 在此完成 Vector4 的结构校验与转换。
    }
}
```

也可以在游戏初始化阶段调用 `TypeConverter.Register(converter)` 显式注册；显式注册的转换器优先，可覆盖默认转换规则。新增业务 `Component` 不需要注册，框架会自动扫描已加载程序集。

生命周期注意事项

`Prefab` 实例化时，`Prefab` 上已有组件的 `Awake/OnEnable` 会先于反射赋值执行；动态 `AddComponent` 也会立即触发该组件的 `Awake/OnEnable`。需要读取配置字段的业务逻辑应放在 `Start` 或更晚阶段，或者在业务侧提供独立的延迟启用机制。这是 Unity 生命周期顺序，不是刷新框架可以对任意组件安全规避的行为。

错误隔离与性能

- JSON 错误：仍按时创建当前对象，但跳过其初始化；后续记录继续。
- `Component` 或字段不存在：输出警告，继续处理其余配置。
- 类型转换或赋值失败：输出错误，继续处理其余字段。
- 调度使用单向 `currentIndex`，不会每帧遍历全部记录。
- 每条 JSON 仅在加载刷新文本时解析一次。
- `Component` 类型、每种 `Component` 的全部 `FieldInfo`、目标类型对应转换器均有缓存。
- `Prefab` 创建和 `GameObject` 字段引用均为字典查询。

文件职责

- `SpawnController.cs`: 注册 Prefab、按时间创建对象、调用初始化入口。
- `SpawnConfigParser.cs`: 解析刷新行、固定列和每条 JSON。
- `JsonParser.cs` / `JsonDataDefine.cs`: 解析标准 JSON 并建立只读配置模型。
- `ComponentInitializer.cs`: 通用组件添加和字段初始化流程。
- `ReflectionCache.cs`: 组件类型发现及 `FieldInfo` 缓存。
- `TypeConverter.cs`: 转换器接口、注册、发现和路由。
- `BuiltInTypeConverters.cs`: 需求规定类型的独立转换器。